Isolamento de recursos com cgroup + namespaces

Objetivo: Aprender a configurar manualmente cgroupsv2, limitando e distribuindo o uso de CPU entre processos, associado ao uso de namespaces para isolamento da tabela de processos e de sistemas de rede.

Parte 1 – Crie um cgroup com limitação de 50% de utilização de CPU

1. Utilizando como modelo o trabalho feito na aula da ultima semana, crie o cgroup e faça a parametrização. Ainda não não execute nenhum processo nele.
2. Execute o comando `ps -ef` no bash para verificar a tabela de processos do SO geral.
3. Use o aplicativo unshare para criar um novo PID namespace e executar um bash dentro deste novo namespace.

   Como root, execute: `unshare -p -f --mount-proc /bin/bash`

4. A princípio, nada mudou, certo? Execute novamente o `ps -ef` e compare o resultado.
5. A partir de uma outra sessão, busque o PID do comando unshare, e considerando que ele é o processo pai do *bash* criado no novo namespace, localize o PID do *bash* e mova-o para dentro do cgroup criado.
6. Execute um `stress --cpu 1` no *bash* criado dentro do namespace e movido para dentro do cgroup, e valide usando o top na outra sessão o quanto de CPU esta sendo consumida (deverá bater com a restrição configurada pelo cgroup).
7. Existe outra forma de fazer isso: voce pode criar o bash no unshare já mapeando-o automaticamente para dentro do cgroup. Pesquise e descubra como fazer isso. Sempre intercale commandos com o `ps -ef` para validar que voce está dentro do namespace.
8. Termine o shell criado pelo unshare e isto destruirá o namespace criado dinamicamente.

Parte 2 – Crie dois namespaces de PID e network, crie interfaces de redes virtuais nos namespaces e conecte-los através de interfaces de rede virtuais.

1. Crie dois novos namespaces completos (PID e rede) usando o comando unshare.
   Em um terminal, crie o primeiro namespace:
   ```
   sudo unshare -pnf --mount-proc /bin/bash
   ```
   Esse será o namespace A.

2. Em outro terminal, repita o comando:
   ```
   sudo unshare -pnf --mount-proc /bin/bash
   ```
   Esse será o namespace B. Não execute nenhum processo nele.

3. Valide que o isolamento de processos está funcionando. Dentro de cada shell criado com unshare, execute:
   ```
   ps -ef
   ```
   Você verá que a lista de processos é muito pequena, pois o namespace de PID mostra apenas o bash atual e seus filhos.

4. Valide que o isolamento de rede também está funcionando. Dentro de cada namespace, execute:
   ```
   ip link
   ```
   Você verá apenas a interface lo (loopback).

5. Ative a interface de loopback:

   ```
   ip link set lo up
   ```

6. Para conectar os dois namespaces, volte ao terminal do host (fora dos unshare) e descubra os PIDs dos bash criados:
   ```
   ps -ef | grep unshare
   ```
   Anote os dois PIDs, um para cada namespace (exemplo: 1234 para nsA e 1235 para nsB).

7. Crie um par de interfaces virtuais veth no host:
   ```
   sudo ip link add vethA type veth peer name vethB
   ```

8. Mova cada ponta do cabo virtual para dentro de um dos namespaces criados:
   ```
   sudo ip link set vethA netns 1234
   sudo ip link set vethB netns 1235
   ```
   (Substitua 1234 e 1235 pelos PIDs reais dos bash criados com unshare.)

9. Configure os endereços IP em cada namespace. Dentro do namespace A (bash criado com unshare de PID 1234, por exemplo):
```
ip addr add 10.40.0.1/24 dev vethA
ip link set vethA up
```

10. Dentro do namespace B:
```
ip addr add 10.40.0.2/24 dev vethB
ip link set vethB up
```

11. Teste a conectividade. Do namespace A, rode:
```
ping -c 3 10.40.0.2
```

12. Do namespace B, rode:
```
ping -c 3 10.40.0.1
```

    Resultado esperado: os dois namespaces devem responder aos pings, mostrando que a comunicação foi estabelecida pela veth.

    Durante todo o exercício, intercale comandos como ps -ef e ip link dentro dos shells para confirmar o isolamento de processos e de rede.

13. Ao terminar os passos de entendimento, descritos abaixo, finalize os bash criados com exit. Isso destruirá automaticamente os namespaces. No host, limpe o par de interfaces caso ainda exista:
```
sudo ip link del vethA 2>/dev/null
sudo ip link del vethB 2>/dev/null
```

Parte 3 - Questões a serem respondidas:

1. O que significam as sintaxes usadas no comando unshare?
2. Existe outra forma de criar um namespace específico de rede. Pesquise e monte a sequência de commandos para o mesmo exercício, usando esta outra forma e explique a diferença.
3. Como deveria fazer para executar algo mais complexo no namespace, ao invés de um simples “bash”? Algo como uma aplicação mais complexa que inicie multiplas threads imediatamente e não te dê tempo de movimenta-as manualmente?

Anexo: Configuração do vagrant para o exercício.

```
Vagrant.configure("2") do |config|
    config.vm.box = "bento/ubuntu-22.04"
    config.vm.hostname = "lab-cgroup"
    config.vm.provider "virtualbox" do |vb|
      vb.gui = true
      vb.memory = "1024"
      vb.cpus = 1
      vb.name = "lab-cgroup"
    end
 end
```